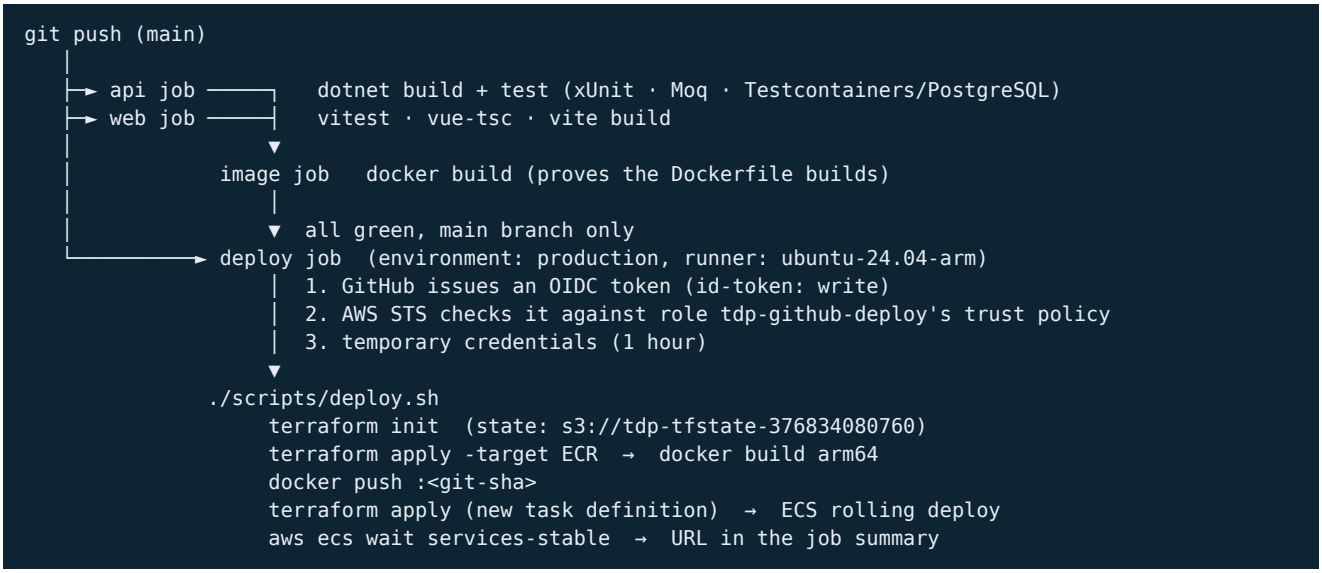
GitHub Actions + Terraform deployment runbook

Transactions Dispute Portal (TheoTechStack/TDP) · how the CI/CD pipeline and AWS infrastructure were set up, step by step, with the exact commands and every problem hit along the way · 20 October 2026

End result. Every push to **main** runs the API tests (xUnit, Moq, Testcontainers), the web tests (Vitest) and a Docker build. If all pass, GitHub Actions signs in to AWS with a short-lived OIDC token (no stored AWS keys) and runs **scripts/deploy.sh**, which builds the ARM64 image, pushes it to ECR and rolls it out to ECS Fargate with Terraform. Terraform state lives in a versioned, locked S3 bucket.

Live app: http://dispute-portal-1381412907.eu-west-1.elb.amazonaws.com

Pipeline at a glance



What lives where

Piece	Location	Role
Workflow	`.github/workflows/ci.yml`	Jobs api, web, image, deploy; triggers, concurrency, OIDC permission.
Deploy script	`scripts/deploy.sh`	One script for CI and laptop: S3 state init, ECR, build/push, apply, wait, job summary.
Bootstrap stack	`infra/bootstrap/main.tf`	Run once from a laptop: S3 state bucket, GitHub OIDC provider, deploy IAM role. Local state.
App stack	`infra/terraform/*.tf`	VPC, ALB, ECS, RDS, ECR, Secrets Manager, IAM, CloudWatch. State in S3.
GitHub settings	Repo → Settings	Public repo, environment production, variable AWS_DEPLOY_ROLE_ARN.

Phase 1 · Tools on the Mac

Step 1 Free disk space

The first **terraform init** failed with *no space left on device* (228 GB of 229 GB used). The AWS provider alone is ~700 MB and Docker builds need several GB. Aim for at least 10 GB free.

```
df -h /                                # check free space
docker builder prune -af               # Docker build cache (safe)
docker image prune -af                 # unused images (re-downloaded when needed)
rm -rf ~/Library/Developer/Xcode/DerivedData
```

Step 2 Install Terraform

brew install failed because the Xcode Command Line Tools were missing, and Homebrew core is frozen at Terraform 1.5.x. Installing HashiCorp's binary directly avoided both:

```
V=1.16.6
curl -fsSLo /tmp/tf.zip   https://releases.hashicorp.com/terraform/$V/terraform_${V}_darwin_arm64.zip
unzip -o /tmp/tf.zip -d /tmp/tf
sudo mkdir -p /usr/local/bin && sudo mv /tmp/tf/terraform /usr/local/bin/
terraform -version                     # Terraform v1.16.6 on darwin_arm64
```

Docker Desktop, the AWS CLI and git were already installed.

Phase 2 · AWS account and credentials

Step 3 Create a dedicated personal AWS account and deploy user

- New AWS account (376834080760), separate from any work account.
- IAM → Users → Create user **tdp-deployer** (no console access) → Attach policies directly → **AdministratorAccess**.
- User → Security credentials → Create access key → *Command Line Interface*. Save the key ID and secret (the secret is shown once). Never create root access keys.

Step 4 Configure a named CLI profile

```
aws configure --profile tdp            # key, secret, region eu-west-1, output json
export AWS_PROFILE=tdp                 # per terminal session
aws sts get-caller-identity            # must show arn:aws:iam::376834080760:user/tdp-deployer
```

> **Region:** new accounts have Cape Town (af-south-1) disabled, so everything runs in **eu-west-1** (Ireland), which is now the project default.

Phase 3 · First deploy from the laptop

Step 5 Validate the Terraform and pin provider versions

```
cd ~/CEO/dispute-portal
terraform -chdir=infra/terraform init      # downloads AWS ~> 6.0 and random ~> 3.6 providers
terraform -chdir=infra/terraform validate  # Success! The configuration is valid.
git add infra/terraform/.terraform.lock.hcl && git commit -m "Pin Terraform provider versions"
```

The lock file must be committed: **deploy.sh** refuses to run with uncommitted changes, so every image tag maps to an exact commit.

Step 6 Run the deploy script

```
TF_VAR_db_backup_retention_days=1 AWS_REGION=eu-west-1 ./scripts/deploy.sh
...
Apply complete!  app_url = "http://dispute-portal-1381412907.eu-west-1.elb.amazonaws.com"
Deployed ae9342…
```

This created 41 resources: VPC, 4 subnets over 2 AZs, internet gateway, security groups, ALB + target group + listener, ECS cluster/service/task definition with autoscaling, RDS PostgreSQL 17, ECR, Secrets Manager JWT key, IAM roles, CloudWatch logs and alarms. State was still a local file at this point.

Phase 4 · Remote state and GitHub OIDC (bootstrap)

Before CI can deploy, two things must exist that CI cannot create for itself: somewhere to keep shared Terraform state, and an AWS role that GitHub is allowed to assume. That is the job of `infra/bootstrap`, run once from the laptop.

Resource	Purpose
S3 bucket tdp-tfstate-376834080760	Terraform state for the app stack. Versioning on, AES-256 encryption, all public access blocked, HTTPS-only bucket policy, prevent_destroy. Locking uses S3-native use_lockfile (no DynamoDB).
OIDC provider token.actions.githubusercontent.com	Lets AWS trust tokens signed by GitHub Actions; audience sts.amazonaws.com.
IAM role tdp-github-deploy	Assumed by the deploy job. Trust policy accepts only this repo's production environment (see Step 12). AdministratorAccess attached, because Terraform manages IAM, VPC and RDS; the control is who can assume it.

Step 7 Apply the bootstrap stack

```
export AWS_PROFILE=tdp
terraform -chdir=infra/bootstrap init
terraform -chdir=infra/bootstrap apply     # type yes
# Outputs:
#   deploy_role_arn = "arn:aws:iam::376834080760:role/tdp-github-deploy"
#   state_bucket    = "tdp-tfstate-376834080760"
```

Step 8 Move the existing state into S3

The app stack's `versions.tf` now declares an S3 backend (bucket and region supplied at init time). Migrating copies the local state into the bucket; nothing in AWS is recreated.

```
terraform -chdir=infra/terraform init -migrate-state \
  -backend-config="bucket=tdp-tfstate-376834080760" \
  -backend-config="region=eu-west-1"          # answer yes to copy
terraform -chdir=infra/terraform state list | wc -l   # 44 = 41 resources + 3 data sources
git add infra/bootstrap/.terraform.lock.hcl && git commit -m "Pin bootstrap provider versions"
```

Phase 5 · GitHub configuration

Step 9 Repository settings

Setting	Value	Why
Visibility	Public	Reviewers can read it; GitHub's ARM64 runners (ubuntu-24.04-arm) are free for public repos.
Environment	production	The deploy job runs in it. Optional required reviewers give a manual approval gate.
Environment variable (production)	AWS_DEPLOY_ROLE_ARN = arn:aws:iam::376834080760:role/tdp-github-deploy	Read by the workflow as vars.AWS_DEPLOY_ROLE_ARN. A variable, not a secret: the ARN grants nothing on its own.
Optional variables	CPU_ARCHITECTURE=X86_64 · DB_BACKUP_RETENTION_DAYS=7	x86 build for a private repo; 7-day backups on a paid AWS account.

Step 10 The workflow (.github/workflows/ci.yml)

Setting	What it does
on: push (main), pull_request, workflow_dispatch	PRs run tests only; main also deploys; deploys can be re-run by hand.
concurrency: cancel-in-progress only for PRs	A newer push never kills a deploy mid-apply (which would leave the state lock held).
deploy.needs: [api, web, image]	Deploy only after every test and the image build pass.
deploy.if: main and not a PR	Pull requests and other branches never deploy.
deploy.environment: production	Puts the environment in the OIDC token and enables the approval gate.
deploy.concurrency: production, no cancel	Deploys queue one after another.
permissions: id-token: write	Allows the job to request an OIDC token from GitHub.
aws-actions/configure-aws-credentials @v4	Exchanges the OIDC token for 1-hour AWS credentials via role-to-assume.
hashicorp/setup-terraform@v3, terraform_wrapper: false	Pins 1.16.6; the wrapper would corrupt terraform output -raw.
run: ./scripts/deploy.sh	Same script as the laptop. Writes the commit and URL to the job summary.

Step 11 Push and watch Actions

```
git push
# GitHub → Actions → CI/CD:  api ✓  web ✓  image ✓  →  Deploy to AWS
```

Phase 6 · First pipeline runs and the two fixes

Step 12 Fix 1: role ARN not visible to the job

Error: `Credentials could not be loaded, please check your action inputs: Could not load credentials from any providers`
Cause: the variable had been saved on an environment called **Prod**, while the job runs in **production**. Environment variables are only visible to jobs in that environment, so role-to-assume was empty.
Fix: add AWS_DEPLOY_ROLE_ARN to the **production** environment (or at repository level), delete Prod, then Re-run failed jobs.

Step 13 Fix 2: GitHub's new immutable OIDC subject

Error: `Could not assume role with OIDC: Not authorized to perform sts:AssumeRoleWithWebIdentity` (after 12 retries)
Cause: since a GitHub change in April 2026, repositories created after 15 July 2026 receive OIDC subjects that include immutable numeric IDs. The trust policy only expected the classic name form.
Fix: look up the IDs and trust both forms, then re-apply the bootstrap.

```
curl -s https://api.github.com/repos/TheoTechStack/TDP
#   .owner.id = 130162778   .id = 1426030960

# infra/bootstrap/main.tf  →  StringEquals token.actions.githubusercontent.com:sub, any of:
#   repo:TheoTechStack@130162778/TDP@1426030960:environment:production   (new, immutable IDs)
#   repo:TheoTechStack/TDP:environment:production                        (classic)

terraform -chdir=infra/bootstrap apply     # 0 added, 1 changed (role trust policy)
git push                                   # new run → Deploy to AWS ✓
```

Pinning to numeric IDs is also stricter: a repository deleted and recreated under the same name cannot assume the role.

All problems hit, and their fixes

Symptom	Cause	Fix
Port 5432 already in use (docker compose)	A local PostgreSQL already listens on 5432.	Compose publishes the DB on 5433 (DB_PORT overrides).
network dispute-portal_default is ambiguous	Leftover duplicate Docker network from a failed start.	docker compose down -v --remove-orphans; remove the networks; up --build.
no space left on device (terraform init)	Mac disk 100% full.	Prune Docker cache/images, delete DerivedData and unused apps.
brew: No developer tools installed	Xcode Command Line Tools missing.	Install Terraform's official binary directly (Step 2).
InvalidClientTokenId (CLI or Terraform)	Stale AWS keys in the default profile or in env vars of an old terminal session.	Use profile tdp, unset AWS_* variables or open a fresh terminal.

Symptom	Cause	Fix
Script not found / chdir: no such directory	Command run from ~ instead of the project folder.	cd ~/CEO/dispute-portal first.
AccessDenied ecr:CreateRepository	AdministratorAccess not attached to tdp-deployer.	IAM → user → Add permissions → AdministratorAccess.
FreeTierRestrictionError (RDS backups)	Free-plan accounts cap backup retention.	Variable db_backup_retention_days; 1 in CI.
Could not load credentials from any providers	Role ARN variable on the wrong environment (Prod).	Variable on production; Re-run failed jobs.
Not authorized: AssumeRoleWithWebIdentity	New immutable-ID OIDC subject format.	Trust both subject forms (Step 13).
Claude listed as a GitHub contributor	Co-Authored-By trailers in commit messages.	Rewrote the messages, recreated the repo as TDP.

Day-to-day operations

Task	How
Ship a change	Commit and **git push** to main. Tests, then automatic deploy; the run summary shows the commit and URL.
Gate deploys	Settings → Environments → production → Required reviewers.
Re-deploy without a code change	Actions → CI/CD → Run workflow (workflow_dispatch) on main.
Manual deploy (fallback)	`export AWS_PROFILE=tdp && TF_VAR_db_backup_retention_days=1 ./scripts/deploy.sh`
Preview infrastructure changes	`terraform -chdir=infra/terraform plan -var image_tag=$(git rev-parse --short=12 HEAD)`
App logs	`aws logs tail /ecs/dispute-portal --since 15m --region eu-west-1`
Find the public URL	EC2 → Load Balancers → dispute-portal → DNS name, or terraform output app_url.
Tear down the app	`terraform -chdir=infra/terraform destroy -var image_tag=x` (bootstrap stays; it costs nothing).
After the assessment	Deactivate the tdp-deployer access key; CI no longer needs it.

Security model in one paragraph

No long-lived AWS credentials exist in GitHub. The deploy job proves its identity with a token GitHub signs for this exact repository and environment, and AWS exchanges it for credentials that expire within an hour. Only the production environment of TheoTechStack/TDP, identified by immutable IDs, can assume the role, so forks, pull requests and other repositories cannot deploy. Terraform state is encrypted, versioned and locked, so concurrent applies cannot corrupt it. The next hardening step is to replace AdministratorAccess on the deploy role with a least-privilege policy scoped to the resources in infra/terraform.

Reference

Item	Value
AWS account / region	376834080760 · eu-west-1
Deploy role	arn:aws:iam::376834080760:role/tdp-github-deploy
State	s3://tdp-tfstate-376834080760/dispute-portal/terraform.tfstate
ECR	376834080760.dkr.ecr.eu-west-1.amazonaws.com/dispute-portal
ECS	cluster dispute-portal · service dispute-portal · logs /ecs/dispute-portal
Load balancer	dispute-portal-1381412907.eu-west-1.elb.amazonaws.com
GitHub IDs	owner TheoTechStack = 130162778 · repo TDP = 1426030960

Source for the OIDC change: GitHub changelog, 23 April 2026, “Immutable subject claims for GitHub Actions OIDC tokens”.